Manual técnico de Proyecto Red

1. Requisitos

- Python 3.10 o superior.
- `matplotlib` es opcional; solo se necesita para generar las gráficas.
- No se requieren dependencias para ejecutar los algoritmos ni las pruebas.

2. Estructura del código

Ruta	Responsabilidad
`red_project/allocation.py`	Reparto óptimo de presupuesto con marginales cóncavas
`red_project/selection.py`	Fuerza bruta, Greedy y cobertura con presupuesto
`red_project/variants.py`	Fallos por rondas y externalidades de vecindad
`red_project/experiments.py`	Instancias, mediciones, estadísticas y gráficas
`red_project/cli.py`	Interfaz de línea de comandos
`tests/test_project.py`	Pruebas automatizadas

3. Instalación opcional

Desde la carpeta raíz del proyecto:

```
python -m pip install matplotlib
```

La instalación puede omitirse si solo se necesitan resultados de consola, archivos CSV o pruebas.

4. Ejecución de demostración

```
python -m red_project.cli demo
```

La demostración ejecuta un reparto de presupuesto, una selección Greedy y la selección exacta sobre una instancia pequeña. La salida incluye la asignación, la utilidad y los nodos seleccionados.

5. Ejecución de experimentos

```
python -m red_project.cli experiment --output results --repetitions 20
```

Parámetros:

- `--output`: carpeta de salida; por defecto `results`.
- `--repetitions`: número de repeticiones por tamaño; por defecto 20.

Las semillas se calculan de forma determinista a partir del tamaño y del número de repetición. Esto permite repetir la medición. La ejecución produce `times.csv` y `summary.csv`; si `matplotlib` está disponible también produce las dos imágenes de resultados.

6. Ejecución de pruebas

```
python -m unittest discover tests -v
```

Las pruebas cubren:

- optimalidad y validaciones del reparto;
- coincidencia de Greedy y exacto en una instancia pequeña;
- factibilidad de la variante con costos;
- casos de cardinalidad cero;
- reproducibilidad de fallos;
- cálculo de externalidades de vecindad.

7. Formatos de entrada y salida de las funciones

`greedy_allocate(marginals, budget)`

`marginals` es una lista de listas. `marginals[i][j]` representa la ganancia de entregar la unidad *j*+1 al nodo *i*. Devuelve `AllocationResult`:

- `amounts`: tupla con unidades asignadas por nodo;
- `utility`: utilidad total.

`greedy_select(node_count, k, utility)`

`utility` recibe un `frozenset` de índices y devuelve un número. Devuelve `SelectionResult`:

- `selected`: tupla ordenada de nodos;
- `utility`: valor de la selección.

`maximum_coverage_utility(coverage)`

`coverage[i]` es un conjunto de elementos cubiertos por el nodo *i*. La función devuelve una utilidad compatible con `greedy_select` y `brute_force_select`.

8. Errores y validaciones

Las funciones lanzan `ValueError` cuando reciben parámetros incompatibles:

- presupuestos negativos o no enteros donde corresponde;
- marginales infinitas, negativas o crecientes;
- k fuera de $0 \le k \le N$;
- costos no positivos o no finitos;
- probabilidades fuera de [0, 1];
- vecindarios con índices inexistentes.

No se deben ocultar estas excepciones: indican que la instancia no cumple el contrato de entrada.

9. Limitaciones conocidas

- La fuerza bruta tiene crecimiento combinatorio $O(C(N, k))$; debe reservarse para instancias pequeñas.
- Los tiempos dependen del equipo, del intérprete y de la carga del sistema.
- La garantía $1 - 1/e$ solo es válida para funciones no negativas, monótonas y submodulares con restricción de cardinalidad.
- La variante con costos y la variante de vecindarios deben interpretarse como heurísticas/modelos separados, salvo que se prueben sus hipótesis específicas.